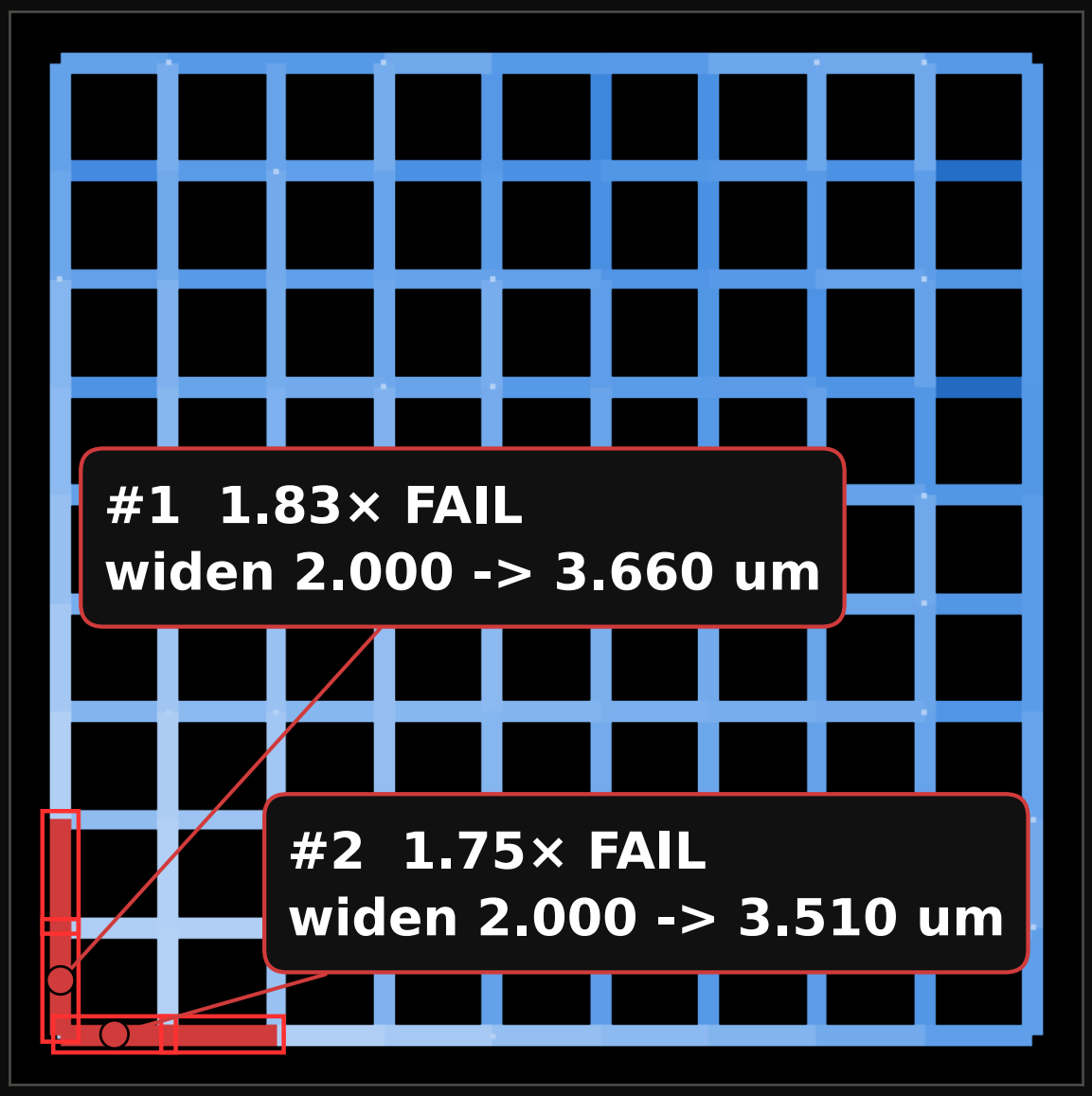
# EM/IR screening your AI agent can run

Electromigration and IR drop on extracted analog layout: every segment checked, the fix in microns or cuts, a report a person can act on.




ok below 0.8×, blue by ratio
warn 0.8× to 1×
fail 1× and over, or not a number
violation marker

agent ▶ `emir next` ▶ `emir run` ▶ `emir report`

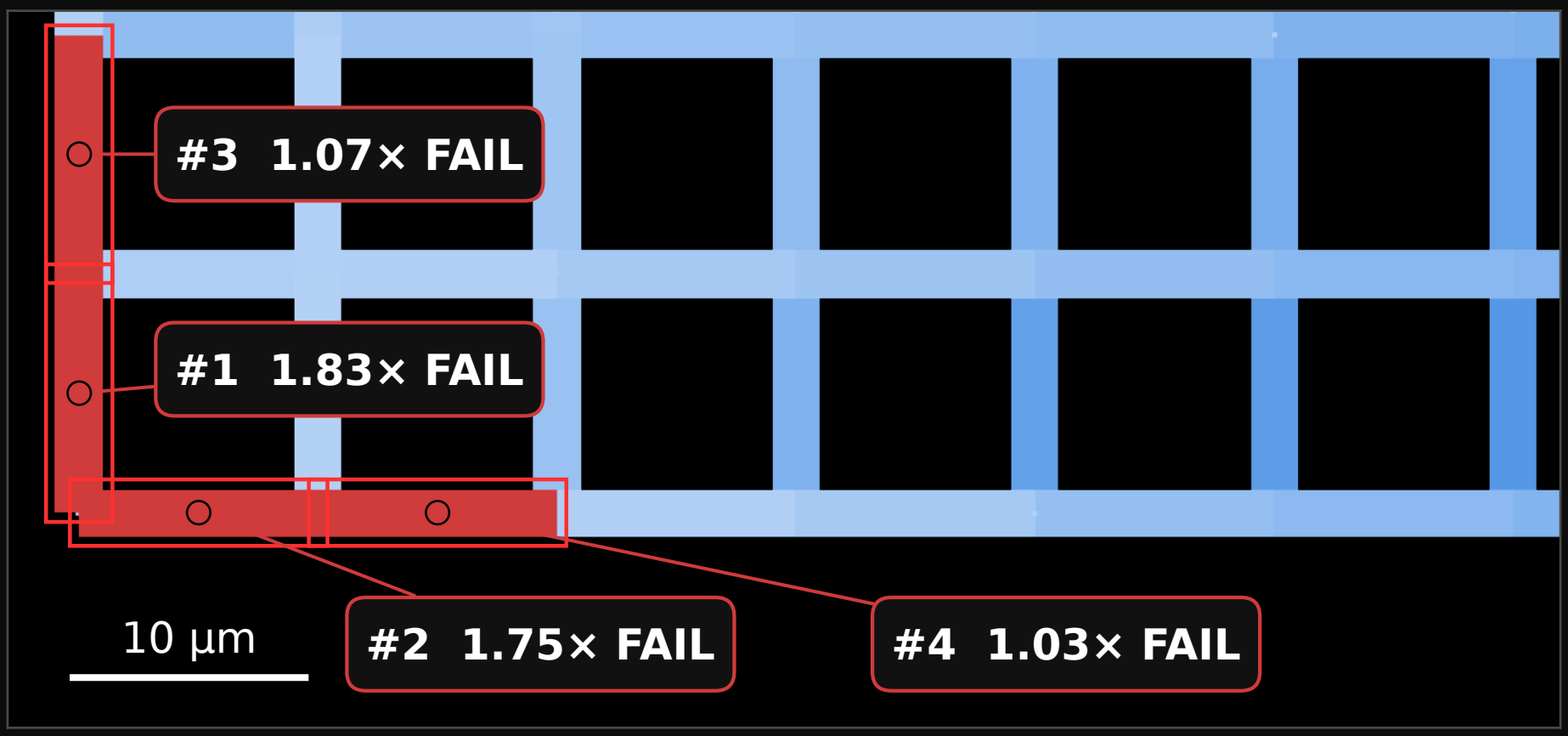
**Screening, not sign-off.**

---

examples/06_supply_mesh: a synthetic 10 x 10 mesh, 32 inverters, Xyce currents, rcc (every metric); invented limits. Swipe for how it works. Screening, not sign-off.

# Every segment, against your limits, with the fix

rcc reads every resistor's current from your RC simulation: each segment's avg, rms and peak, the IR drop, the fix. Here: where the supply enters the mesh.




ok below 0.8×, blue by ratio
warn 0.8× to 1×
fail 1× and over, or not a number
violation marker

| by hand | the run |
|---|---|
| **Kirchhoff at the feed**<br>3.656 + 3.510 + 0.231 = 7.397 mA<br>#1 up, #2 right, the corner inverter | **7.397 mA**<br>the devices' average current |
| **#1, the strap up from the feed**<br>3.656 mA / 2.0 µm / 1.0 mA/µm = 1.83×<br>average current over width, over the limit | **1.83× FAIL**<br>fix: widen 2.000 -> 3.660 um |

examples/06_supply_mesh, invented round-number limits; Xyce currents, rcc: every branch current read, every metric checked. Screening, not sign-off.

# Built to be driven by an AI agent

Every command answers in JSON, every failure carries a code, and the tool itself says what to run next.

**19**
commands, each with --format json

**210**
stable codes: branch on the code, not the text

**26**
JSON schemas (emir schema)

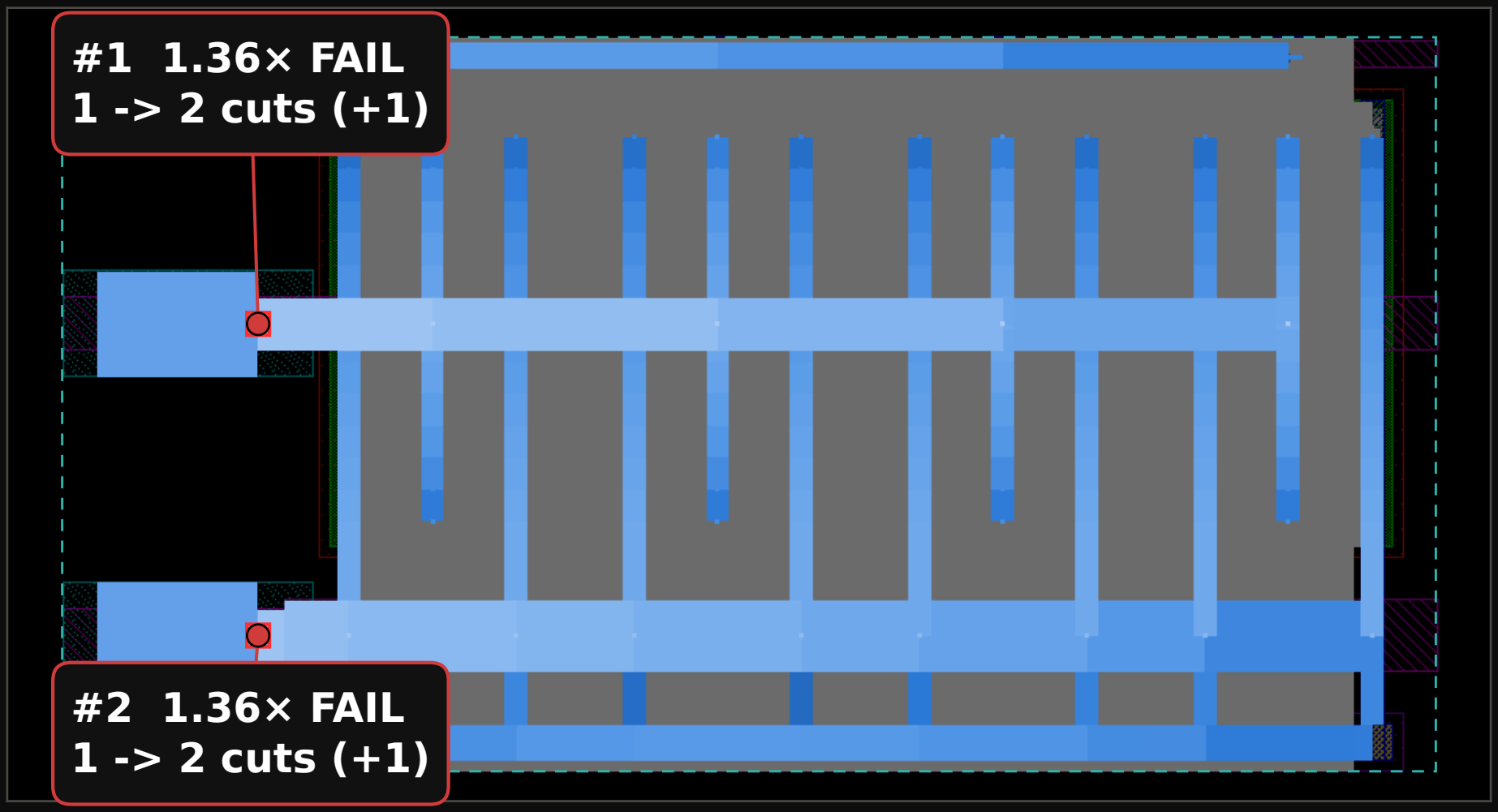
```
$ emir next --dspf loadrail.dspf --process …
mode: rcc -- currents from a simulation of the RC …
  1. emir selftest
  2. emir nets loadrail.dspf
  3. emir verify-sim --sim xyce
  4. emir simulate tb_rcc.cir --sim xyce --dspf …
  5. emir check loadrail.dspf --process …
  6. emir run loadrail.dspf --process process.yaml …
  7. emir report --from emir_out/summary.json

$ emir run loadrail.dspf --process process.yaml …
  overall verdict: FAIL -- 1 net(s): 1 FAIL, 0 …
$ emir report --from emir_out/summary.json
  Verdict: FAIL -- 1 net(s): 1 FAIL, 0 WARN, 0 CLEAN;
    worst 1.042x on VDD.
  IR drop VDD: 28.068 mV (from the feed); dynamic …
```

The plan and its steps on examples/02_xyce, run as printed (scripted; rcc; Xyce currents, the bundled illustrative limits); long lines shortened (…). Screening, not sign-off.

# Open PDK, open tools: magic extracts, rcc checks

magic, with the public geoout patch, writes every resistor's layer, width and place; Xyce simulates that RC netlist; EMIR reads every branch current.




ok below 0.8×, blue by ratio
warn 0.8× to 1×
fail 1× and over, or not a number
violation marker

**1135**
resistors in magic's network, every one read

**1.36×**
via2 avg: one cut where two are needed

**6.69 mV**
IR drop on out (4.84 mV on vss)

---

examples/08_open_layout: a synthetic SKY130 cell (four NMOS), magic with the geoout patch, Xyce, rcc; the limits of pdks/sky130.yaml; diffusion, poly and wells grey: no limit, not checked. Screening, not sign-off.

# Your limits and tools, verified where they run

Point your agent at the repository: it reads AGENTS.md and proves the install first.

| | |
|---|---|
| **Limits** | pdks/ for SKY130, GF180MCU and IHP SG13G2, every value cited; or typed by hand from your design manual (emir process new, emir process check) |
| **Cadence** | emir extract plans PVS LVS and Quantus; Spectre save lines; a Virtuoso Explorer highlights the worst segments, read-only |
| **Simulators** | Spectre, HSPICE, Xyce, ngspice: each passes emir verify-sim on your machine before EMIR relies on it |

**Start** `emir selftest` ▶ `emir next` ▶ `emir run` ▶ `emir report`

**Agents** Claude Code, Codex, any MCP client (emir mcp)

**Code** github.com/skycadeda/emir · MIT licence

**Screening, not sign-off.**

Foundry-qualified EM/IR remains the sign-off flow; EMIR says where to look first, and what it did not check. A research project, with no guarantee of accuracy: test and verify its results.

---

Spectre, HSPICE, Quantus, PVS and Virtuoso need licences: none runs in this repository's tests, and each is UNVERIFIED until checked on site. Every number in these pages comes from a run in the repository (scripts/make_figures.py).